

# PROJECT REPORT

## *Basic Study Planner: A Command-Line Task Management System*

### ABSTRACT

The Basic Study Planner is a lightweight, Python-based Command-Line Interface (CLI) application designed to assist students and professionals in managing their academic or personal schedules. The system integrates task creation, persistent local storage, cross-threaded background notifications, and active time-tracking utilities (stopwatch and timer). This report details the architectural design, core modules, implementation techniques, and technical limitations of the software.

## 1. Introduction

In an era dominated by resource-intensive Graphical User Interfaces (GUIs) and web-based applications, there remains a distinct need for lightweight, fast, and distraction-free productivity tools. The Basic Study Planner was conceptualized to fulfill this need by providing a terminal-based environment where users can manage their study routines without leaving their command-line workspace.

The primary objective of this project is to implement a fully functional scheduling system using only the Python Standard Library. By avoiding external dependencies, the application ensures high portability and easy setup. Key functionalities include CRUD (Create, Read, Update, Delete) operations for study tasks, intelligent background alarms that alert the user asynchronously, and focus-enhancing time management tools.

## 2. System Architecture

The application follows a modular architecture, segregating data management, background services, and user interface into three distinct Python files. This separation of concerns ensures that the codebase remains maintainable, scalable, and easy to debug.

The system relies on a local JSON file (`data.json`) acting as the persistence layer, which is manipulated exclusively through a dedicated data modeling class. Concurrently, a daemon thread monitors system time against the loaded data to trigger real-time notifications.

### 3. Module Description

The software is divided into three primary modules, each responsible for a specific domain of the application's logic.

| Module Name                   | Primary Classes/Functions                                              | Responsibility                                                                                                                                                                                                             |
|-------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>models.py</code>        | <code>Task</code><br><code>ScheduleManager</code>                      | Acts as the Data Layer. It defines the structure of a single study task and handles the serialization/deserialization of task arrays to and from the <code>data.json</code> file. It encapsulates all file I/O operations. |
| <code>alarm_service.py</code> | <code>AlarmService</code>                                              | Handles business logic for background alerts. It instantiates a daemon thread that continuously polls the current system time and compares it against active tasks, printing an alert if a match occurs.                   |
| <code>main.py</code>          | <code>main()</code><br><code>menu()</code><br><code>stopwatch()</code> | The Presentation Layer. Contains the interactive CLI loop, handles user input/validation, prints color-coded menus using ANSI escape sequences, and executes the timer/stopwatch functionalities.                          |

### 4. Core Features & Implementation

#### 4.1. Persistent Task Management

The system provides full CRUD capabilities. When a user adds a task via `main.py`, the `ScheduleManager` converts the inputs into a `Task` object and immediately commits it to `data.json`.

- **Dynamic Indexing:** When a task is deleted, the `delete_task` method filters out the requested ID and then automatically iterates through the remaining tasks to assign them new, sequential IDs (starting from 1). This ensures the UI always presents a clean, contiguous list to the user.
- **Input Normalization:** Days of the week are converted to lowercase, and times are strictly formatted as HH:MM to ensure consistent data matching. The input parser also supports the "everyday" keyword, expanding it into a full 7-day array.

## 4.2. Asynchronous Alarm System

To provide notifications without halting the main user interface loop, the application utilizes Python's `threading` module.

### Daemon Threading

The `AlarmService.start_monitoring()` method launches a thread with `daemon=True`. This ensures that the background monitoring gracefully terminates the moment the main program (the CLI menu) exits, preventing orphaned background processes.

### Duplicate Alert Prevention

Because the thread checks the time every 15 seconds, a task scheduled for 14:30 could theoretically trigger up to four times within that minute. To prevent this, a `triggered_tasks` set temporarily logs a unique composite key (`task_id_day_time`) for the alarm. This set is automatically flushed when the system clock rolls over to the next minute.

## 4.3. Active Time Management (Timer & Stopwatch)

Beyond scheduling, the application aids active studying through terminal-based time trackers utilizing the `time.monotonic()` function for high-resolution, hardware-level time tracking unaffected by system clock updates.

- **Countdown Timer:** Accepts hours, minutes, and seconds, calculating a total duration. It uses `\r` (carriage return) and ANSI code `\033[K` to overwrite the current line, creating a dynamic, ticking clock effect in the terminal without spamming new lines.
- **Stopwatch:** Counts upwards from zero. It utilizes the `msvcrt` module to continuously poll for keyboard interrupts (specifically the Enter key) to stop the timer accurately.

## 5. Technical Limitations

---

While the application is fully functional, it possesses a few inherent technical constraints based on its current design:

1. **Platform Dependency:** The stopwatch feature strictly relies on `msvcrt`, a Microsoft Visual C/C++ Runtime Library module. Consequently, executing the stopwatch option on Linux or macOS environments will trigger an `ImportError`.
2. **Mutable Task IDs:** Because IDs are re-indexed upon deletion to maintain UI aesthetics, they do not serve as reliable primary keys over the long term. If a user memorizes ID "3" for "Math", deleting ID "1" will shift "Math" to ID "2".

## 6. Future Scope & Enhancements

---

The foundational architecture of the Basic Study Planner allows for several avenues of future expansion:

- **Cross-Platform Compatibility:** Replacing the `msvcrt` dependency with a cross-platform alternative (such as the `keyboard` library or standard `sys.stdin` configurations) would make all features universally accessible.
- **Immutable UUIDs:** Transitioning from sequential integer IDs to Unique Universal Identifiers (UUIDs) on the backend while keeping display numbers separate would ensure database integrity.
- **Audible Notifications:** Integrating libraries like `playsound` to provide audible chimes when an alarm triggers or a timer finishes, improving the utility of background alerts.

## 7. Conclusion

---

The Basic Study Planner successfully demonstrates the capability to build a robust, multi-threaded productivity application using standard Python libraries. By prioritizing a lightweight footprint and user-friendly command-line interface, it serves as an effective tool for time management and task scheduling. The modular codebase establishes a strong foundation for future iterative improvements and feature expansions.